

Homework 2

“Asynchronous JavaScript and XML/JSON”

The following problem set is worth 100 points total. Please submit a zip file containing all relevant code to the Canvas drop box by the due date of the assignment.

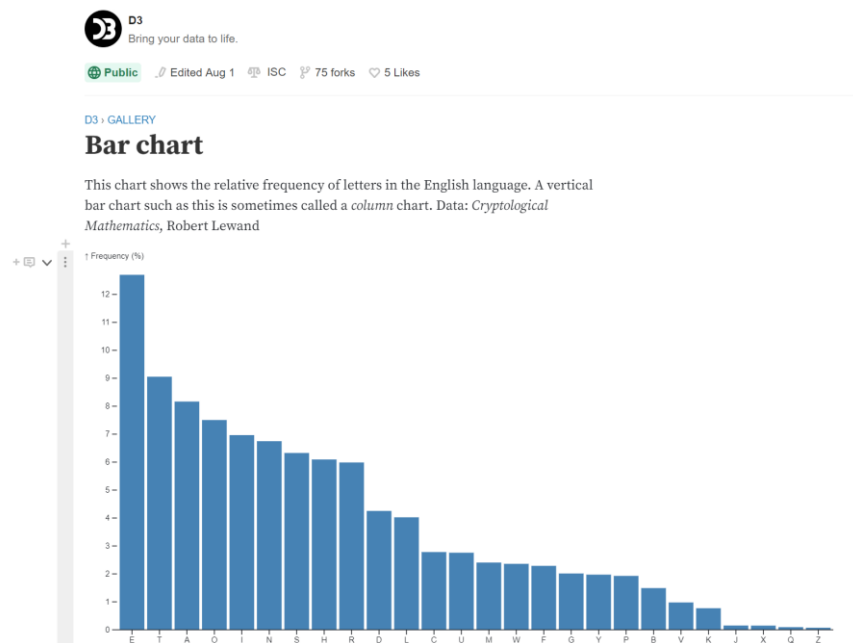
Requirements

The purpose of this assignment is to familiarize you with retrieving data for JavaScript applications from external data sources. You will also be incorporating a JavaScript chart engine called **D3.js** into your applications.

D3.js

D3 is a graphics library for producing dynamic visualizations using JavaScript. It's used heavily in data science. Many organizations use it in their applications, one being the New York Times. You can see some interesting examples here (many links unfortunately are paywalled.)

<https://www.nytimes.com/spotlight/graphics>

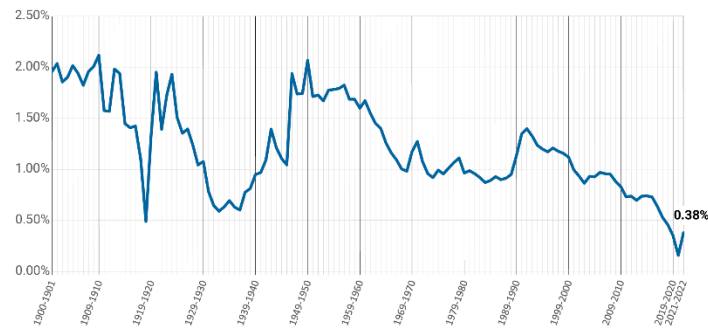


For a quick start on how to use D3.js, here is an excellent tutorial: <https://d3js.org/getting-started>. So, with all of this out of the way, we will be building a few graphs for this assignment. And at the core, we need data.

Population Chart (20pts)

Using D3.js, produce an application which generates a **line chart** which shows the population of the United States year over year. As an example of something you can generate:

FIGURE 1
US annual population growth, 1900-01 to 2021-22*



*July 1 to July 1 of each year

Source: William H. Frey analysis of U.S. Census Bureau historical population estimates, including 2020-22 annual estimates released December 22, 2022



For a barebones data set, you can use this link:

<https://datausa.io/api/data?drilldowns=Nation&measures=Population>. If you want to be more creative however, you can query the **United States Census** with their API. Available APIs (and sample calls) can be found here: <https://www.census.gov/data/developers/data-sets.html>

Word Cloud (20pt)

A word cloud is a visualization of text data. As an example:



D3.js can generate these. For this part, you will create an application which asks the user for a word. You will then query an API for all the **synonyms** of the word and then generate a word cloud. As to the API you can use, the following exists: <https://www.wordsapi.com/>.

NOTE: this is a free API for up to 2500 requests per day. You will need an API key. You can include the API key you created in your Canvas submissions. Be careful about this when developing your application.

Weather Map (30pt)

Some end points might involve you creating an account to obtain a free API key. For this problem, I want you to create me a weather forecast screen similarly to how Bing does it:



You can retrieve data for your screen from **weather.gov**. Their API is documented here:

<https://www.weather.gov/documentation/services-web-api>

To retrieve the forecast data you'll need to make a call such as the following:

<https://api.weather.gov/gridpoints/TOP/31,80>

The response is a JSON data set containing information like the following:

```

    },
    - maxTemperature: {
      sourceUnit: "F",
      uom: "unit:degC",
      - values: [
        - {
          validTime: "2019-02-01T15:00:00+00:00/PT11H",
          value: 11.111111111111143
        },
        - {
          validTime: "2019-02-02T13:00:00+00:00/PT13H",
          value: 11.111111111111143
        },
        - {
          validTime: "2019-02-03T13:00:00+00:00/PT13H",
          value: 14.444444444444514
        },
        - {
          validTime: "2019-02-04T13:00:00+00:00/PT13H",
          value: 3.3333333333333712
        },
        - {
          validTime: "2019-02-05T13:00:00+00:00/PT13H",
          value: 0
        },
        - {
          validTime: "2019-02-06T13:00:00+00:00/PT13H",
          value: -0.5555555555555429
        },
        - {
          validTime: "2019-02-07T13:00:00+00:00/PT13H",
          value: -5
        },
        - {
          validTime: "2019-02-08T13:00:00+00:00/PT13H",
          value: -1.6666666666666288
        }
      ]
    },
  ],
},

```

The endpoint breaks up data into *zones*. To retrieve the forecast zones you'll need for the above API call, you'll need to first make an API call to retrieve the zone data via latitude and longitude coordinates:

<https://api.weather.gov/points/39.7456,-97.0892>

```

properties: {
  @id: "https://api.weather.gov/points/39.7456,-97.0892",
  @type: "wx:Point",
  cwa: "TOP",
  forecastOffice: "https://api.weather.gov/offices/TOP",
  gridX: 31,
  gridY: 80,
  forecast: "https://api.weather.gov/gridpoints/TOP/31,80/forecast",
  forecastHourly: "https://api.weather.gov/gridpoints/TOP/31,80/forecast/hourly",
  forecastGridData: "https://api.weather.gov/gridpoints/TOP/31,80",
  observationStations: "https://api.weather.gov/gridpoints/TOP/31,80/stations",
  - relativeLocation: {
    type: "Feature",
    - geometry: {
      type: "Point",
      - coordinates: [
        -97.086661,
        39.679376
      ]
    }
  },
},

```

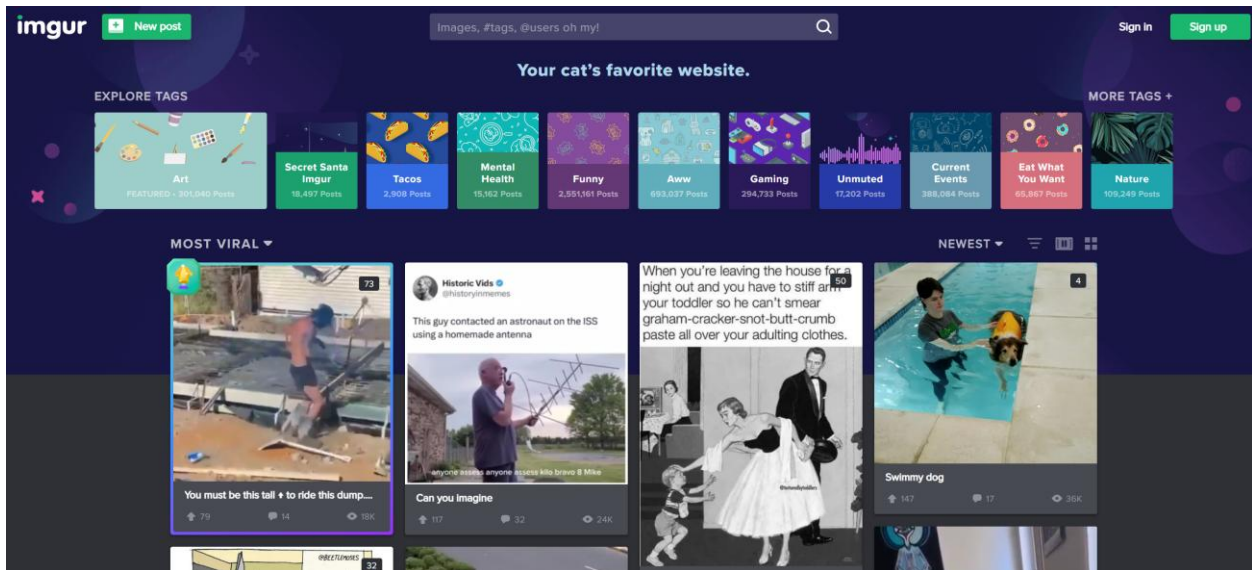
For your screen, the user should be able to change your weather grid by entering their zip code. You'll need to convert their zip code into latitude and longitude coordinates. *MapQuest* has an open API you can use for that data as well, particularly their **Geocoding API**:

<https://developer.mapquest.com/>

Feel free to use any image assets in your solution to your liking.

Imgur Client (30pt)

The popular image sharing web application **Imgur** has a free application you can use to access anything: <https://apidocs.imgur.com/?version=latest>.



Using their API, produce a client which does the following:

- Grabs images from a random gallery
- Lists statistics about each image such as the number of likes and number of comments.
- When someone clicks on the image, retrieve each of the comments and display on a detail page.

Submission Requirements

For the submission, submit a zip file with all assets needed for me to run your four applications. Feel free to use any technique used in class (either DOM manipulation or jQuery.) Submit using the Canvas drop box before the due date.